Basic Branching & Merging

Tuesday, November 25, 2025 10:50 PM

Let’s go through a simple example of branching and merging with a workflow that you might use in the real world. You’ll follow these steps

1. Do some work on a website.
2. Create a branch for a new user story you’re working on.
3. Do some work in that branch.

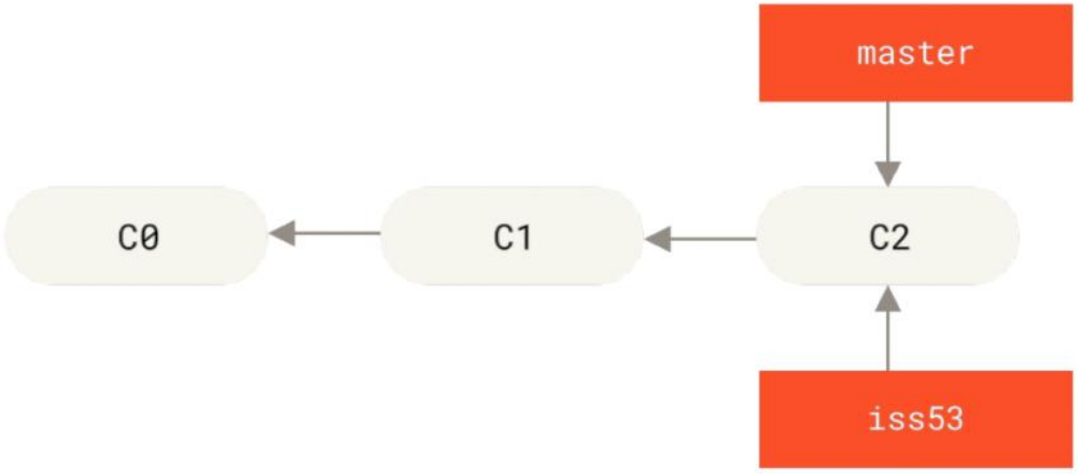
At this stage, you’ll receive a call that **another issue** is **critical** and you **need a hotfix**. You’ll do the following:

1. Switch to your production branch.
2. Create a branch to add the hotfix
3. After it’s tested, merge the hotfix branch, and push to production.

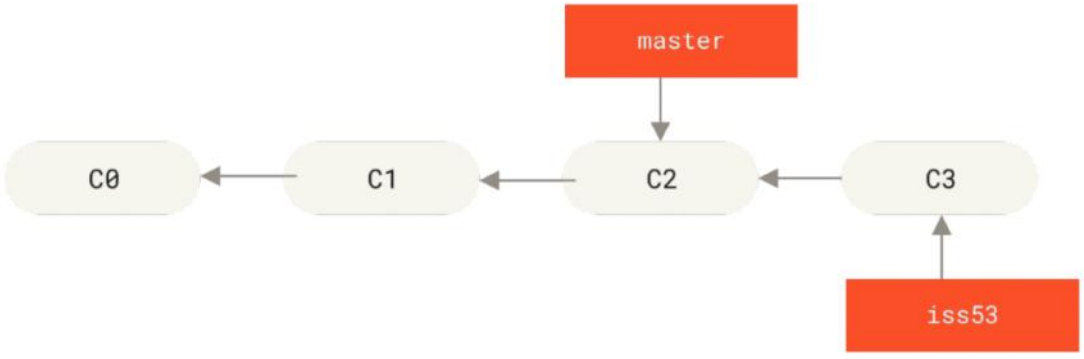
Basic Branching

Assume, you are working on a project that already have couple of commits on master branch

Let's say, you decided to work on a certain issue named 'iss53'



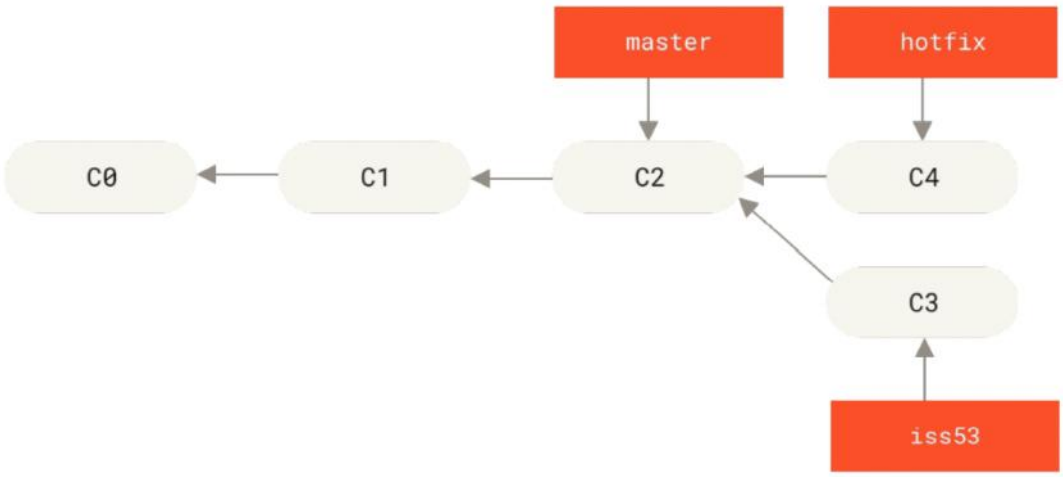
You did changes and do some commits. Doing so moves the **iss53** branch **forward**, because you have it checked out (that is, your HEAD is pointing to it):



<u>At this stage, **assume** your project manager called and **discussed** about **an issue** which needs to be **fixed immediately**.</u>

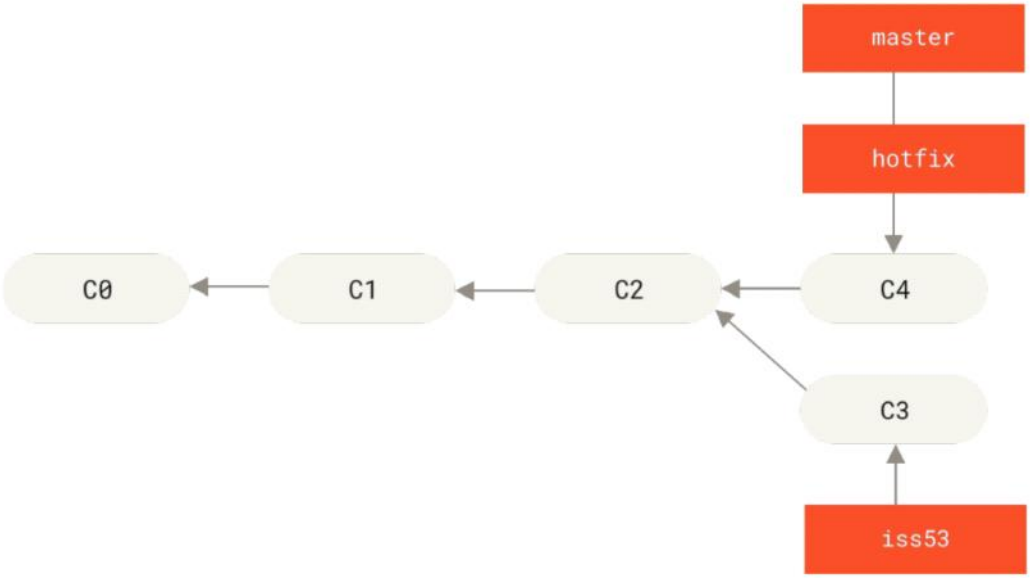
Question: How you will proceed?

1. Switch back to master
2. Create new branch with name hotfix or any other name you like
3. Fix the issue
4. Commit changes



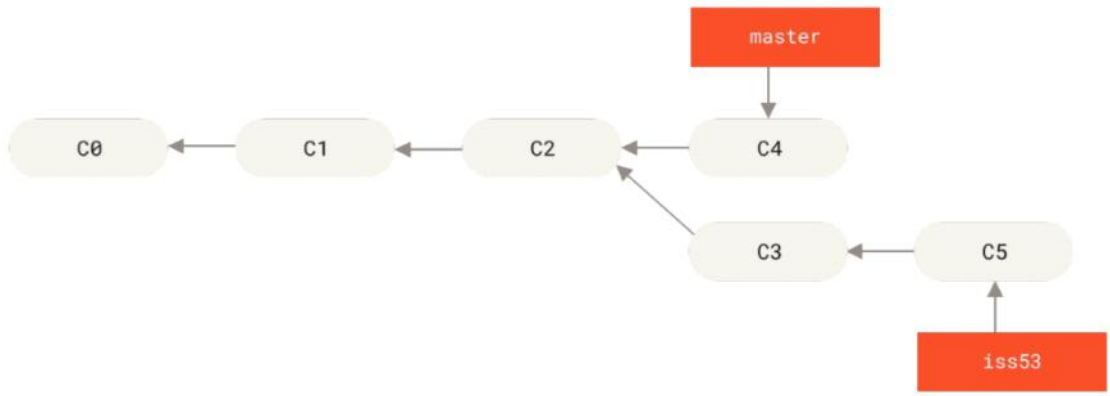
<u>You did some **tests** and **verified** the issue has been **fixed** and ready for **deployment**</u>

1. Checkout to master
2. Merge the hotfix with master
    a. *git merge hotfix*
3. Push changes



<u>Now, you are done with hotfix, now what ?</u>

1. Delete hotfix branch, as you no longer needed it
    a. *git branch -d hotfix12*
    b. *git branch --delete hotfix12*
2. Switch back to the iss53 branch and do the changes
    a. *git checkout iss53*

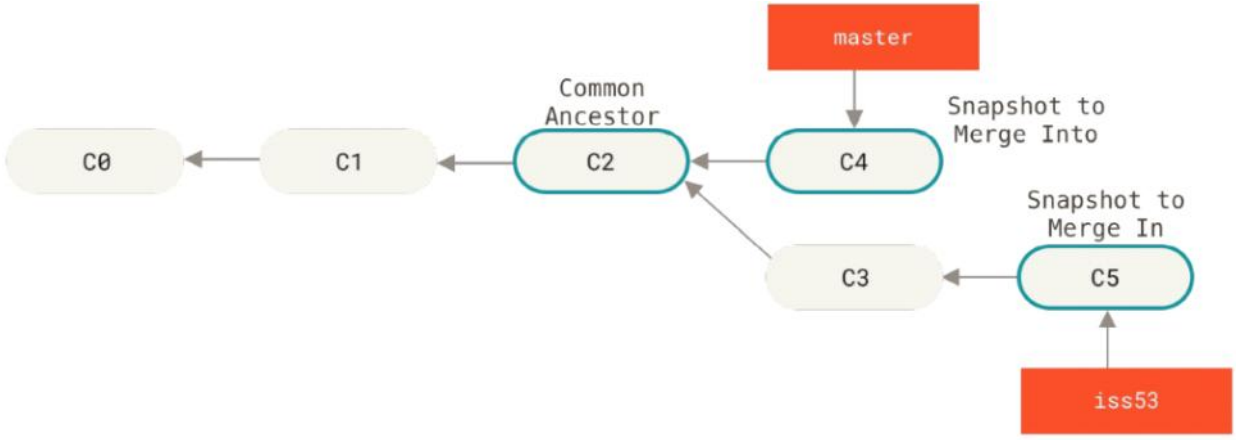


Basic Merging

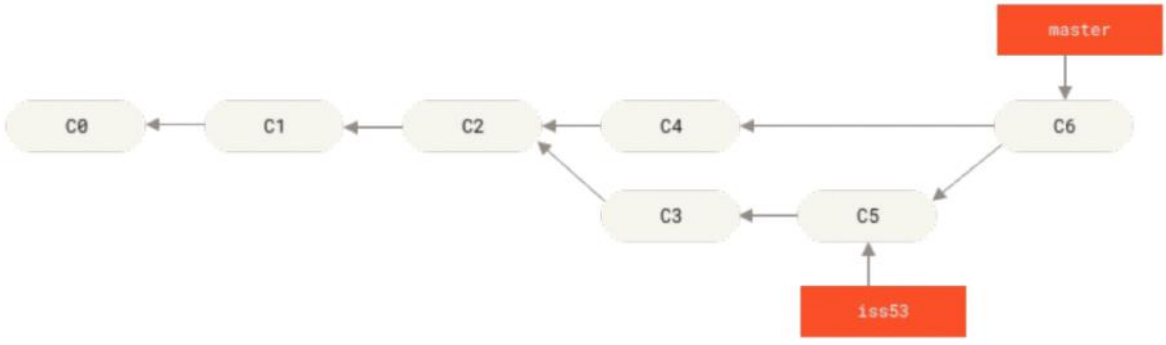
Assume, you are done with changes on 'iss53' branch and **ready** to **merge** with master branch

1. Checkout to master branch
2. Merge iss53 into master
    a. *git merge iss53*

Note: This is a **special** commit, known as **merge commit**. Git does a simple, **three-way merge** using the **two snapshots** pointed to by the **branch tips** and the **common ancestor** of the two.



Instead of just **moving** the **branch pointer forward**, Git creates a **new snapshot** that results from this three-way merge and automatically **creates a new commit** that points to it.



Basic Merging Conflicts

Occasionally, this merging process doesn't go smoothly.

If you **changed** the **same part** of the **same file** differently in the **two branches** you're merging, Git won't be able to merge them cleanly.

If your fix for iss53 **modified** the **same part** of a file as the **hotfix branch**, you'll get a **merge conflict** that looks something like this:

```
$ git merge iss53
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
```

1. Git will pause merging
2. You can see the unmerged files by running the following command
   a. *git status*
3. Git adds standard **conflict-resolution markers** to the files that have conflicts

```
<html>
<head>
    <title>
    This is a devops class
    </title>
</head>

<body>
<<<<<<< HEAD
    Hello, every one, stay tunned, we will be back soon
=======
    <p> Dear Students, welcome to the DevOps class </p>
>>>>>>> iss53
</body>
</html>
```

4. This means the version in **HEAD** (your master branch, because that was what you had checked out when you ran your merge command) is the **top part** of that block (everything above the =======)
5. Resolve these conflicts
   a. Can open files in any editor and decide which to block to keep
6. Once done, run *git add <files>*